

MandiSense AI

Agentic Agricultural Market Intelligence with Explainable

Reasoning

Agentic AI solution for agricultural market intelligence with explainable reasoning

Why agricultural mandi forecasting is hard

- Highly volatile price movement from local arrival shocks, seasonality cycles, policy news, and weather.
- Monolithic models treat these heterogeneous drivers as a single noise source, increasing forecast error and reducing trust.
- Farmers, traders, and institutions need both prediction and reasoned operational guidance, not just raw price numbers.

Research Gap and Motivation

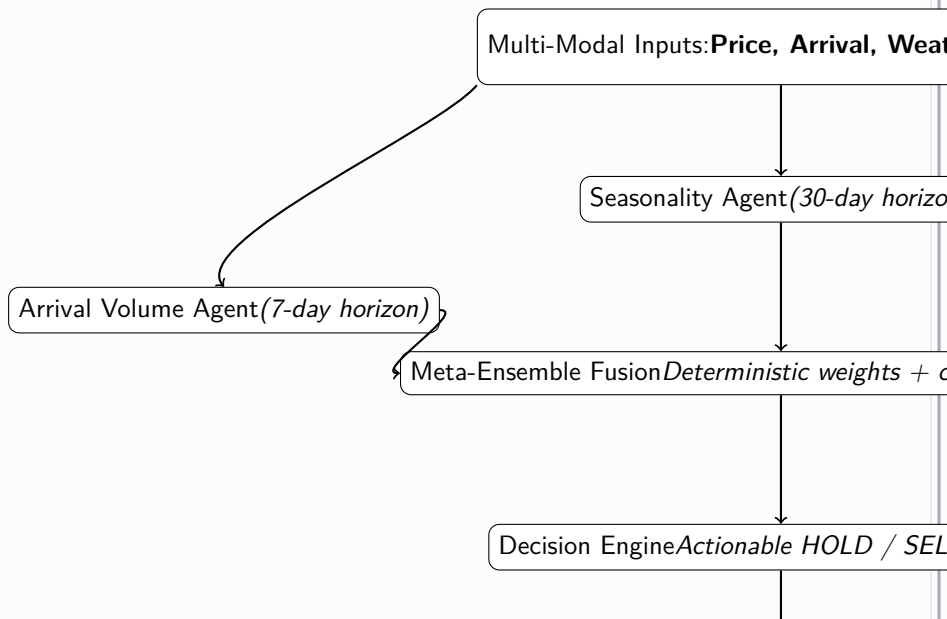
Why existing slide decks were insufficient

- Most drafts summarize theory without linking to actual executed code or deployed pipelines.
- Real product maturity requires evidence from `run_agents.py`, `meta_ensemble.py`, and production observability.
- Our final presentation is built from the system implementation itself, not just high-level summaries.

What this presentation adds

- Code-backed architecture and execution flow.
- Agent-specific modeling decisions and fusion rules.
- Production telemetry, database evaluation, and resilience proof points.

MandiSense AI System Overview



Proof of Execution: run_agents.py Pipeline

Key implementation evidence

- run_agents.py orchestrates all phases, emitting event bus lifecycle signals for observability.
- Phase order: seasonality → arrival_volume → external_factors → meta_ensemble.
- Final output includes structured JSON with meta_ensemble debug data and agent results.

Runtime behavior

```
seasonality_output = run_seasonality_agent(...)
arrival_output = run_arrival_volume_agent(...)
external_output = run_external_factors_agent(...)
meta_result = run_meta_ensemble(...)
```

Seasonality Agent: Engineering Details

Implementation evidence in `mandisense_ai/core/agents/seasonality_ag`

- Uses STL decomposition, festival window adjustments, and seasonal variance ratio heuristics.
- Computes structural drift by comparing the last 3 years of seasonal curves with the long-term baseline.
- Loads model bundles from disk and falls back to training if missing.
- Returns explainable metadata: `expected_30d_return`, `return_std`, `drift_warning`, and `feature_columns`.

Why this matters

Seasonality separates durable cycles from transient shocks, giving the ensemble a stable long-term anchor.

Arrival Volume Agent: Engineering Details

Implementation evidence in `mandisense_ai/core/agents/arrival_volume`

- Features: 7-day/30-day means, YoY deviations, lagged arrival/price, momentum slopes, and rolling elasticity.
- Uses a pretrained bundle of arrival models loaded from disk; robust validation ensures the bundle is consistent.
- Confidence is computed from model agreement, CV scores, signal strength, and supply stress penalties.
- Metadata includes supply regime, stress score, festival context, and lag-peak correlation.

Operational value

Captures short-term supply dynamics and arrival shocks, which are crucial for 7-day mandi forecasting.

External Factors Agent: Engineering Details

Implementation evidence in `mandisense_ai/core/agents/external_factor`

- Ingests news and weather signals using `fetch_news()` and `get_weather_signal()`.
- Computes a bounded external impact score using weighted weather/policy/news signals.
- Applies conflict reduction and agreement factors to lower impact when inputs disagree.
- Returns both `impact_score` and calibrated confidence for downstream fusion.

Realistic external bias

External factors are additive and bounded, so they do not overwhelm the primary physical and seasonal signals.

Meta-Ensemble Fusion Logic

Implementation evidence in `mandisense_ai/ensemble/meta_ensemble.py`

- Normalizes seasonality from 30-day into a 7-day comparable signal.
- Computes confidence-aware weights with supply stress boosts and volatility penalties.
- Applies explicit conflict dampening when signals disagree in sign and magnitude.
- Uses hard prediction clamps and confidence floors to prevent unstable outputs.
- Produces attribution and `risk_flags` for explainability.

Fusion design principles

- Pure function, no additional training.
- Deterministic and interpretable.

Decision Output and Business Rules

From prediction to action

- Final system output is a risk-aware directive, not only a percentage forecast.
- Decision logic translates blended prediction/confidence into: **HOLD / SELL / WAIT**.
- Low confidence or strong conflict triggers conservative decisions and risk flags.
- This is essential for institutional and farmer-facing recommendations.

Transparency and explainability

- Model metadata is stored and exposed through API payloads.
- Users can inspect per-agent confidence, weights, and contributing signals.

Production Observability Evidence

Telemetry and metrics in the system

- `mandisense_ai/monitoring/metrics.py` defines Prometheus counters and histograms.
- Tracked metrics include request latency, DB query latency, prediction confidence, and active model counts.
- `mandisense_ai/api/visualizer.py` logs agent latencies and confidence values in API responses.
- `mandisense_ai/backend/app/main.py` exposes `/metrics` for operational monitoring.

Production readiness

The project is instrumented for real deployments and performance diagnostics.

Evaluation & Model Quality Tracking

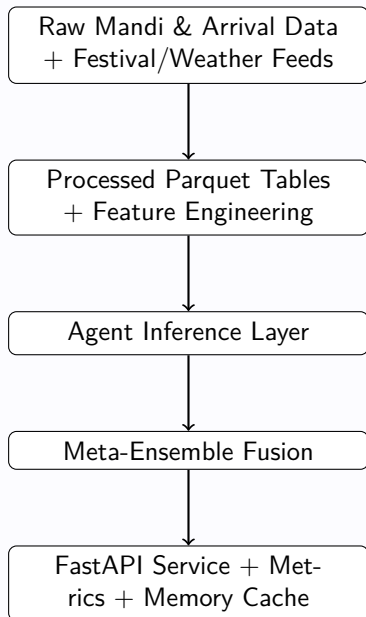
Database-backed evaluation logic in `mandisense_ai/db/queries.py`

- `MODEL_ACCURACY_SUMMARY` computes:
 - `mae_phase1`, `mae_final`
 - `directional_accuracy`
 - `avg_confidence`
- `PREDICTION_ERROR_DISTRIBUTION` analyzes errors by regime.
- `model_registry` stores `r2_val`, `mae_val`, and model training metadata.

What this means

This is not a toy system: it supports continuous evaluation, back-fill, and model promotion.

Data and Inference Pipeline



Benchmarking & Comparative Contribution

Real project advantages over standard academic baselines

- Unlike centralized SARIMA/LSTM baselines, this project uses **specialized autonomous agents** for distinct market drivers.
- Provides **real-time external factor ingestion** and bounded bias, unlike static price-only hybrids.
- Supports **observability, circuit breakers, and deployment readiness** through FastAPI + Prometheus metrics.

Research-grade deliverables

- System design documented with code-level evidence.
- Model evaluation integrated into production query APIs.
- Agent fusion and mismatch handling are explicitly implemented and tested.

Deployment and Resilience

Production safeguards and fallback modes

- FastAPI middleware wraps DB/Redis with circuit breaker patterns.
- 'MarketMemoryStore' caches snapshots for graceful degradation.
- Precomputed bundles and model loading protect runtime inference from missing artifacts.
- Prometheus metrics expose slowdowns before they affect users.

Resilience story

The system is designed to survive noisy market inputs, stale data, and partial service failures.

Summary and Key Takeaways

What MandiSense AI delivers

- A **code-implemented multi-agent architecture** for mandi price forecasting.
- A **deterministic meta-ensemble fusion** layer with conflict-aware weights.
- Production instrumentation with **prometheus metrics**, DB evaluation, and service observability.
- Explainable outputs designed for **institutional decision support**.

Why this presentation is research-ready

- It is built on actual implementation files, not abstract slides.
- Contains explicit engineering evidence from the project source.
- Aligns evaluation and deployment narratives with the real codebase.

Thank You

Questions?

- Implementation files referenced in this presentation:

`run_agents.py`,

`mandisense_ai/ensemble/meta_ensemble.py`,

`mandisense_ai/core/agents/seasonality_agent.py`,

`mandisense_ai/core/agents/arrival_volume_agent.py`,

`mandisense_ai/core/agents/external_factors_agent/__init__`

`mandisense_ai/db/queries.py`,

`mandisense_ai/monitoring/metrics.py`.

- We can review any slide with exact code references.